



T.C.
**MARMARA
UNIVERSITY**



FACULTY OF ENGINEERING
COMPUTER ENGINEERING DEPARTMENT

CSE4088 Introduction to Machine Learning
Term Project - Model Evaluation Report

Project Title

Email Spam Detection

Group#7 Members

Emir Büçkün - 150119024

Mehmet Sina Çağlar - 150123821

Fatma Melisa Küçük - 150119916

İrem Kubalas - 150119721

Ayşenur Tüfekçi - 150119699

Instructor

Assoc. Prof. Murat Can Ganiz

Table of Contents

1. Introduction.....	3
2. Evaluation Metrics.....	3
3. Model Performance.....	3
3.1. Initial Results: Model Performance Overview.....	3
3.2. Optimized Model Performance Metrics.....	5
3.3. Computational Efficiency of Models.....	8
4. Optimization Techniques.....	8
4.1. Hyperparameter Optimization.....	8
Grid Search.....	8
Randomized Search.....	9
Comparison and Effectiveness.....	10
5. Error Analysis.....	10
5.1. Random Forest Classifier.....	10
5.2. Logistic Regression.....	10
5.3. Naive Bayes.....	10
5.4. K-Nearest Neighbors.....	10
6. Recommendations.....	11
6.1. Address Class Imbalance.....	11
6.2. Feature Engineering.....	11
6.3. Enhance Optimization.....	11
6.4. Integrate Error Analysis.....	11
6.5. Model Selection Criteria.....	11
7. Conclusion.....	12
8. References.....	12

1. Introduction

The purpose of this report is to evaluate the performance of the machine learning models developed during the spam detection project. Evaluation was conducted using predefined metrics to compare the efficacy of the models and identify areas for improvement. The evaluation process also considers the computational efficiency and effectiveness of the optimization techniques applied.

2. Evaluation Metrics

The following metrics were used to assess the models:

- **Accuracy:** The proportion of correctly classified instances.
- **Precision:** The ratio of true positive predictions to the total positive predictions.
- **Recall:** The ratio of true positive predictions to all actual positive instances.
- **F1-Score:** The harmonic mean of precision and recall, balancing their trade-offs.

3. Model Performance

The evaluation of the machine learning models was conducted to understand their ability to classify spam messages effectively. This process aimed to compare the strengths and weaknesses of different algorithms and identify the most suitable one for spam detection. By analyzing the models' metrics and computational efficiency, we sought to balance predictive performance with practical implementation concerns. This section provides an overview of the initial performance results, the impact of optimization techniques, and the computational efficiency of the models.

3.1. Initial Results: Model Performance Overview

3.1.1 Random Forest Classifier

	Precision	Recall	F1-Score
0 (ham)	0.88	0.93	0.91

1 (spam)	0.80	0.70	0.75
Accuracy			0.86
Macro Avg	0.84	0.81	0.83
Weighted Avg	0.86	0.86	0.86

Table 3.1: Initial Results for Random Forest Classifier

The Random Forest Classifier achieved an accuracy of 86% with a precision of 80% for the positive class. As shown in Table 3.1.1, the recall for the positive class is 70%, indicating that the model struggles to detect some spam messages. This suggests that while the model performs well on negative classes, it underperforms slightly on the positive class.

3.1.2 Logistic Regression

	Precision	Recall	F1-Score
0 (ham)	0.80	0.95	0.87
1 (spam)	0.76	0.43	0.55
Accuracy			0.79
Macro Avg	0.78	0.69	0.71
Weighted Avg	0.79	0.80	0.78

Table 3.1.2: Initial Results for Logistic Regression

Logistic Regression showed an accuracy of 79%, but as seen in Table 3.1.2, the recall for the positive class is only 43%. This low recall indicates that the model fails to capture complex relationships in the data, likely due to its linear assumptions.

3.1.3 Naive Bayes

	Precision	Recall	F1-Score
0 (ham)	0.80	0.58	0.68
1 (spam)	0.39	0.65	0.48
Accuracy			0.60

Macro Avg	0.59	0.62	0.58
Weighted Avg	0.68	0.60	0.62

Table 3.1.3: Initial Results for Naive Bayes

The Naive Bayes model achieved an accuracy of 60%. Table 3.1.3 indicates a recall of 65% and a precision of 39% for the positive class. These metrics reveal that the model struggles with classification consistency, likely due to its independence assumptions not aligning with the dataset's dependencies.

3.1.4 k-Nearest Neighbors

	Precision	Recall	F1-Score
0 (ham)	0.80	0.90	0.85
1 (spam)	0.66	0.46	0.55
Accuracy			0.78
Macro Avg	0.73	0.68	0.70
Weighted Avg	0.76	0.78	0.76

Table 3.1.4: Initial Results for k-Nearest Neighbors

K-Nearest Neighbors (KNN) delivered an accuracy of 78%. However, Table 3.1.4 shows a recall of 46% for the positive class, suggesting that the model suffers from performance degradation when handling high-dimensional datasets.

3.2. Optimized Model Performance Metrics

3.2.1 Random Forest Classifier (Grid Search)

	Precision	Recall	F1-Score
0 (ham)	0.87	0.94	0.90
1 (spam)	0.82	0.64	0.72
Accuracy			0.86
Macro Avg	0.85	0.79	0.81

Weighted Avg	0.85	0.86	0.85
---------------------	------	------	------

Table 3.2.1: Optimized Results for Random Forest Classifier (Grid Search)

3.2.2 Random Forest Classifier (Randomized Search)

	Precision	Recall	F1-Score
0 (ham)	0.88	0.94	0.91
1 (spam)	0.82	0.69	0.75
Accuracy			0.87
Macro Avg	0.85	0.81	0.83
Weighted Avg	0.86	0.87	0.86

Table 3.2.2: Optimized Results for Random Forest Classifier (Randomized Search)

The Random Forest Classifier, optimized using Grid Search, achieved an accuracy of 86% with a recall of 69% for the positive class, as detailed in Table 3.2.1. When optimized with Randomized Search, the model's accuracy improved to 87%, and the recall for the positive class increased to 75%, as shown in Table 3.2.2. These results confirm that hyperparameter optimization significantly enhances the model's performance.

3.2.3 Logistic Regression

	Precision	Recall	F1-Score
0 (ham)	0.81	0.95	0.87
1 (spam)	0.78	0.44	0.56
Accuracy			0.80
Macro Avg	0.79	0.69	0.72
Weighted Avg	0.80	0.80	0.78

Table 3.2.3: Optimized Results for Logistic Regression

Logistic Regression improved slightly after optimization, reaching an accuracy of 80% and a recall of 44% for the positive class. Table 3.2.3 illustrates that, despite

optimization, the model's performance on the positive class remains limited, likely due to class imbalance.

3.2.4 Naive Bayes

	Precision	Recall	F1-Score
0 (ham)	0.80	0.58	0.68
1 (spam)	0.39	0.65	0.48
Accuracy			0.78
Macro Avg	0.59	0.62	0.58
Weighted Avg	0.68	0.60	0.62

Table 3.2.4: Optimized Results for Naive Bayes

The Naive Bayes model showed no improvement after optimization, maintaining an accuracy of 60%. Table 3.2.4 demonstrates that this model remains unsuitable for the dataset, as it fails to capture the complex dependencies.

3.2.5 k-Nearest Neighbours

	Precision	Recall	F1-Score
0 (ham)	0.81	0.92	0.86
1 (spam)	0.72	0.47	0.57
Accuracy			0.79
Macro Avg	0.76	0.70	0.72
Weighted Avg	0.78	0.79	0.78

Table 3.2.5: Optimized Results for k-Nearest Neighbors

K-Nearest Neighbors achieved an accuracy of 79% with a recall of 47% for the positive class. However, as shown in Table 3.2.5, the improvements are minor, and the model's limitations in handling high-dimensional data persist.

3.3. Computational Efficiency of Models

Model	Training Time (sec)	Optimization Time (sec)
Random Forest	1.83	257.28
Logistic Regression	0.05	0.98
Naive Bayes	0.00	0.05
k-Nearest Neighbors	0.00	0.30

Table 3.3: Computational Efficiency of Models

Table 3.3 compares the computational efficiency of the models. Random Forest required the most resources due to its complexity, while Logistic Regression and Naive Bayes were highly efficient. k-Nearest Neighbors had minimal training time but required more computation during prediction.

4. Optimization Techniques

4.1. Hyperparameter Optimization

Hyperparameter optimization was applied systematically to enhance model performance by fine-tuning key parameters for each algorithm. This process involved the following techniques:

Grid Search

Grid Search is a brute-force optimization technique that systematically evaluates all possible combinations of specified hyperparameter values. For this project:

- **Random Forest:** Key hyperparameters such as the number of trees (`n_estimators`), the maximum depth of each tree (`max_depth`), and the minimum number of samples required to split a node (`min_samples_split`)

were tested. Grid Search ensured a comprehensive exploration of the parameter space.

- **Logistic Regression:** Parameters such as the regularization strength (`C`) and the penalty type (L1 or L2) were evaluated. These parameters directly influence the model's ability to generalize.
- **Naive Bayes:** Although Naive Bayes typically has fewer hyperparameters, variations in the smoothing parameter (`alpha`) were tested to improve performance on this dataset.
- **K-Nearest Neighbors (KNN):** Key parameters such as the number of neighbors (`n_neighbors`), the distance metric (`metric`), and the weighting function for distances (`weights`) were explored. These parameters significantly affect the classification accuracy.

Grid Search provided a thorough and systematic approach, ensuring that all hyperparameter combinations were tested exhaustively for each model. However, this method is computationally expensive, especially for models with large parameter spaces like Random Forest and KNN.

Randomized Search

Randomized Search is a more efficient alternative to Grid Search, as it randomly samples a predefined number of hyperparameter combinations from the search space. This technique was applied to the Random Forest model, which has a larger number of hyperparameters and is computationally intensive. The key benefits and results include:

- **Efficiency:** Unlike Grid Search, Randomized Search explores the parameter space more quickly by limiting the number of combinations tested.
- **Key Hyperparameters Tested:** For Random Forest, the same parameters as in Grid Search (`n_estimators`, `max_depth`, `min_samples_split`) were considered, but with broader value ranges.
- **Results:** Randomized Search yielded marginally better results compared to Grid Search. It achieved higher accuracy (87%) and improved recall for the positive class (75%), as detailed in section 3.2.2.

Comparison and Effectiveness

- **Grid Search:** Best suited for smaller parameter spaces where exhaustive evaluation is feasible. It provided reliable results for simpler models like Logistic Regression and Naive Bayes.
- **Randomized Search:** More effective for complex models like Random Forest, where the parameter space is vast. It reduced computational costs while maintaining or slightly improving model performance.

These hyperparameter optimization techniques were crucial in achieving the best possible performance for each model, particularly for the Random Forest Classifier, which demonstrated the most significant improvement through Randomized Search.

5. Error Analysis

5.1. Random Forest Classifier

False Positives: Some ham messages were misclassified as spam, potentially due to overlapping feature distributions.

False Negatives: Spam messages with subtle patterns were missed, suggesting a need for additional feature engineering.

5.2. Logistic Regression

Low Recall: The model struggled to identify spam messages, likely due to the imbalanced dataset and linear assumptions.

5.3. Naive Bayes

Feature Independence Assumption: Failed to capture complex dependencies, leading to poor overall performance.

5.4. K-Nearest Neighbors

Dimensionality Sensitivity: Performance degradation occurred due to high-dimensional data and class imbalance.

6. Recommendations

6.1. Address Class Imbalance

The dataset's imbalanced nature, with far fewer spam messages compared to ham, impacted model performance. To address this, oversampling techniques like SMOTE or class weighting during model training can be implemented. These methods would help the models better focus on the minority class (spam) and improve recall.

6.2. Feature Engineering

The inclusion of additional features such as sender metadata, spam-specific keywords, and semantic analysis could improve the model's ability to detect subtle patterns in spam messages. Advanced techniques like word embeddings or TF-IDF vectorization could also be explored for better representation of textual data.

6.3. Enhance Optimization

Training and optimization processes, especially for computationally intensive models like Random Forest, could benefit from techniques such as early stopping and incremental learning. This would reduce training times while maintaining high performance.

6.4. Integrate Error Analysis

Modify the evaluation process to focus more on error-prone cases, particularly spam messages misclassified as ham. Penalizing such misclassifications in the cost function could guide the models to better performance.

6.5. Model Selection Criteria

Establish clear criteria for selecting the final model, focusing on the balance between predictive performance and computational efficiency. This ensures practicality in real-world deployment scenarios.

7. Conclusion

The evaluation process identified Random Forest as the most effective model for email spam detection due to its balanced performance in metrics such as accuracy, precision, recall, and F1-score. Optimization techniques like Grid Search and Randomized Search further enhanced its effectiveness, highlighting the importance of hyperparameter tuning.

Logistic Regression demonstrated potential but suffered from low recall, which limits its application in critical spam detection tasks. Similarly, k-Nearest Neighbors provided moderate performance but was affected by dimensionality and class imbalance. Naive Bayes, while computationally efficient, struggled with the dataset's complex dependencies.

Future work should address class imbalance, improve feature engineering, and explore advanced optimization methods to further enhance model robustness. This project lays a strong foundation for developing reliable and efficient spam detection systems.

8. References

- *Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, 2011.*
- *"Imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning," Journal of Machine Learning Research, 2017.*